

Sprawozdanie z projektu

Nazwa przedmiotu

Sieci komputerowe

Rok akademicki

Kierunek

Prowadzący

Semestr

Sekcja

2025/26

IPpp

Dr inż. Łukasz Maliński

4

1

Skład sekcji

Kacper Skowroński

Igor Juraszek

Podział obowiązków

Kacper	Igor
Code review - zarządzanie repozytorium oraz zatwierdzanie zmian, Przygotowywanie prezentacji z etapów pracy, Serializacja binarna danych, Przygotowanie prezentacji	Naprawa błędów i testowanie aplikacji, Walidacja projektu, Implementacja nowych funkcji w GUI, Przygotowanie prezentacji

Opis działania protokołu komunikacyjnego, serializacji i wykrywania gubienia pakietów

Architektura i role w protokole komunikacyjnym

Komunikacja między węzłami sieciowymi opiera się na protokole TCP/IP przy wykorzystaniu asynchronicznych gniazd biblioteki Qt. Aplikacja może działać w trybie Serwera lub Klienta, przyjmując logiczną rolę Regulatora lub Obiektu. Komunikacja jest w pełni sterowana zdarzeniami, co zapobiega blokowaniu głównego wątku interfejsu graficznego.

Struktura ramki i proces serializacji

Aby zapewnić niezależność od architektury sprzętowej i spójność danych, zaprojektowano uniwersalną strukturę Ramka, która jest serializowana binarnie przy użyciu QDataStream.

Strumień danych zawsze rozpoczyna się od pierwszego bajtu identyfikującego typ ramki, na podstawie którego system rozpoznaje rodzaj ładunku i dynamicznie go deserializuje.

Ramka danych symulacji					
qint32	qint32	uint	double	double	double
4 bajty	4 bajty	4 bajty	8 bajtow	8 bajtow	8 bajtow
typ ramki	aktualny stan	numer probki	u	y	w

Ramka PID				
qint32	qint32	double	double	double
4 bajty	4 bajty	8 bajtow	8 bajtow	8 bajtow
typ ramki	typ calki	kp	ti	td

Ramka ARX													
qint32	qint32	double	qint32	double	int	double	double	double	double	double	double	bool	bool
4 bajty	4 bajty	8 bajtów * rozmiar wektora A	4 bajty	8 bajtów * rozmiar wektora B	4 bajty	8 bajtów	8 bajtów	8 bajtów	8 bajtów	8 bajtów	8 bajtów	1 bajt	1 bajt
typ ramki	rozmiar wektora A	dane wektora A	rozmiar wektora B	dane wektora B	opóźnienie	amplituda szumu	u max	u min	y max	y min	czy limit	czy szum	

Ramka GWZ						
qint32	qint32	double	double	int	double	double
4 bajty	4 bajty	8 bajtów	8 bajtów	4 bajty	8 bajtów	8 bajtów
typ ramki	typ sygnału	amplituda	okres	interwał	wypełnienie	składowa stała

Aby uodpornić aplikację na fragmentację strumienia, na warstwie odczytu zastosowano mechanizm buforowania transakcyjnego. Aplikacja przetwarza ramkę dopiero wtedy, gdy w buforze znajduje się jej pełna struktura.

Wykrywanie opóźnień, gubienia pakietów i jitteru

Protokół TCP sprzętowo gwarantuje dostarczenie i kolejność pakietów, co sprawia, że zgubienie pakietu objawia się w aplikacji jako drastyczny wzrost opóźnienia z powodu retransmisji. Aby zapobiec desynchronizacji symulacji, zaimplementowano własny mechanizm monitorowania na warstwie aplikacji:

Numeracja i pomiar: Regulator wysyłając ramkę nadaje jej inkrementowany identyfikator oraz zapisuje dokładny czas jej nadania przy użyciu obiektu `QElapsedTimer`. Obiekt ARX, odsyłając odpowiedź, zachowuje ten sam numer próbki, co pozwala regulatorowi na wyliczenie faktycznego opóźnienia transportowego sieci po jej powrocie.

Monitorowanie zaległości: System na bieżąco analizuje wielkość kolejki niepotwierdzonych próbek. Jeśli odpowiedzi docierają z opóźnieniem, liczba elementów w kolejce rośnie. System wizualizuje stan łącza za pomocą wskaźnika (kolor zielony oznacza brak opóźnień (maksymalnie 1 niepotwierdzona próbka), pomarańczowy sygnalizuje pojawienie się opóźnień, od 2 do 5 oczekujących próbek, a czerwony ostrzega o poważnym opóźnieniu powyżej 5 próbek). Brak odpowiedzi jest również oznaczany na głównym wykresie symulacji za pomocą czerwonej kropki.

Mechanizm Fail-safe: W tle działa mechanizm kontrolujący maksymalny dopuszczalny czas oczekiwania. Jeśli najstarsza wysłana ramka nie uzyska odpowiedzi przez czas przekraczający limit 5000 ms, system klasyfikuje połączenie jako utracone. Gniazdo TCP jest natychmiast asynchronicznie zamykane, a symulacja powraca do trybu lokalnego.

Historia rozwoju aplikacji

Etap 1:

- Wdrożenie do aplikacji,
- Weryfikacja funkcjonalności,
- Odszukanie celowych błędów zostawionych przez poprzednią grupę,
- Wstępne zaprojektowanie ramki protokołu sieciowego,
- Implementacja wstępnego modelu komunikacji w celu nawiązania połączenia i przestania pakietów testowych między instancjami.

Etap 2:

- Poprawa znalezionych wcześniej błędów,
- Rozbudowa interfejsu graficznego o okno konfiguracji trybu sieciowego,
- Podział pętli symulacyjnej na role.

Etap 3:

- Zaprojektowanie końcowej wersji protokołu sieciowego,
- Wdrożenie warstwy diagnostycznej: pomiar czasu odpowiedzi, monitorowanie kolejki niepotwierdzonych próbek oraz mechanizm fail-safe rozłączający martwe połączenia,
- Wdrożenie mechanizmu binarnej serializacji.

Etap 4:

- Usunięcie błędu powodującego pokrywanie się wykresów mimo opóźnień,
- Dodanie wizualizacji zgubionych pakietów na wykresach,
- Zmiana wykresu ciągłego na schodkowy styl rysowania wykresów,
- Implementacja modułu sztucznego opóźnienia.

Napotkane trudności

- Problem z brakiem opóźnienia

Testowanie komunikacji sieciowej odbywało się początkowo w obrębie jednej maszyny lub w sieci lokalnej. W takich warunkach protokół TCP przysyłał pakiety praktycznie natychmiastowo. Przez brak naturalnych opóźnień transportowych nie dało się przetestować, jak układ zareaguje na jitter czy opóźnione ramki. Należało zaimplementować mechanizm wymuszający sztuczną degradację łącza. W interfejsie dodano suwak opóźnienia sieci. Przetwarzanie odebranej ramki jest sztucznie wstrzymywane na zadaną liczbę milisekund.

- Problem z jednakowymi wykresami mimo opóźnień

Mimo że pakiety docierały z opóźnieniem, wykresy wyjścia z obiektu nadal rysowały się w sposób idealny. Wynikało to z tego, że funkcja rysująca po prostu czekała na ostateczny wynik i rysowała zaktualizowany punkt na osi czasu bez pokazania, co działo się w międzyczasie. Konieczne było oddzielenie lokalnej estymaty wypracowanej przez wewnętrzny model regulatora od rzeczywistego sygnału, który faktycznie przychodził z sieci z opóźnieniem.

Podsumowanie - czego się nauczyliśmy

Kacper	Igor
- Praca na istniejącym kodzie - Zrozumienie działania protokołów sieciowych - Zrozumienie i implementacja poprawnej serializacji podczas wysyłania danych przez sieć	- Praca na istniejącym kodzie - Projektowanie struktury ramki danych, - Implementacja binarnej serializacji i deserializacji - Zrozumienie jak jitter, utrata pakietów wpływają na UAR